Password_Manager

- data
  - {} passwords.json
- file_handler.py
- main.py
- password_operations.py

main.py

```
from password_operations import add_password, view_password
, delete_password, generate_password
from file_handler import save_passwords_to_file,
    load_passwords_to_file.

passwords = load_passwords_to_file()

def show_menu()
    print ("\n Passwords Manager,")
    print ("1. Enter 1 to add password:")
    print ("2. Enter 2 to View password:")
    print ("3. Enter 3 to Delete Password:")
    print ("4: Enter 4 to generate Passwords:")

while True:
    show_menu().
    choice = input("Enter your choice:")

    if choice == "1":
        add_password(passwords).
    elif choice == "2":
        view_password(passwords)
    elif choice == "3":
        delete_password(passwords)
    elif choice == "4":
        generate_password(passwords)
```

```
elif choice == "5":
        save_passwords_to_file(passwords)
        print("password is saved successfully")
else:
    print("In Invalid choice").
```

password_operations.py

```
import random
import string
def add_password(passwords):
    special_chars = string.punctuation

    user_website = input("please Enter Your website Name")
    .strip() → (1)

    while True:
        user_name = input("please Enter your User Name")
        .strip()
        if user_name.replace(" ", "").isalpha()
            break                                  ↓
        else:                                      (2)
            print("Invalid entry. User Name must
                be aplhabetic order").

    while True:
        user_passwords = input("Enter your password:")
        .strip()

        if len(user_password) < 8:
            print("password must be at leart 8 chaeacter,
                long. try agenin").

        if not any(char in special_chars for char in
        user_passwords); ↓ (3)
```

```
            print("password must contain atleast one digit.
                  try again").
            continue

        print("password is valid.")
        break
    password = {
        "website": user_website,
        "username": user_name,
        "password": user_password
    }
    passwords.append(password)

    print("your website, username and password are
          successfully saved").

def view_password(passwords):
    if not passwords:
        print("No Records available.")
        return

    print("In Password List")
    for users in passwords:
        print(f"WEBSITES: {users['website']} |
        USERNAME: {users['username']} | PASSWORD:
        {users['password']}")

def delete_password(passwords):
    view_password(passwords)
    print("In Enter the name of user you want to
          remove password.")
    try:
        input_username = str(input("enter the username"))
```

PAGE NO.....
DATE / /

```
for queryname in passwords:
    if queryname['username'] == input_username
        passwords.remove(queryname)
        print(f"Removed Password {queryname['password']} and username {queryname['username']}")
        return
    else:
        print("Please enter matching username")
except ValueError:
    print("Please enter alphabetic order username")
```

```
def generate_password(passwords):
    lowercase_letters = string.ascii_lowercase
    uppercase_letters = string.ascii_uppercase
    digits = string.digits
    special_chars = string.punctuation

    all_chars = lowercase_letters + uppercase_letters
                + digits + special_chars

    password_length = 12

    while True:
        generated_password = ''.join(random.choice(all_chars)
                             for _ in range(password_length))
```

```
    if (any(char in lowercase_letters for char in
    generated_password) and any(char in uppercase_letters
    for char in generated_password) and any(char
    in digits for char in generated_password) and
    any(char in special_chars for char in generated_
        password)):
        break

user_website = input("please Enter Your Websites Name.")
.strip()
while True:
    user_name = input("please Enter your Name").strip()
    if user_name.replace(" ", "").isalpha():
        break
    else:
        print("Invalid entry. User Name
              must be alphabetic order")

passwords.append({"website": user_website,
                  "username": user_name, "password":
                      generated_password})

print(f"Password is generated is: {generated_
      password}")
```

Question (1) strip()

(2) isalpha()

(3) How this line works

(4) loops/remove only remove username or val dict of that username.

(5) How this line works

File_handler.py

```
import json
import os

FILE_PATH = "data/passwords.json".

def save_passwords_to_file(passwords)
    try:
        with open(FILE_PATH, "w") as
            file
                json.dump(password, file)
    except Exception as e:
            print("Password saved to file.")
    except Exception as e:
            print(f"Error saving passwords:
                {e}).
def load_password_to_file()
    try:
        if not os.path.exists(FILE_PATH):
            return[]
        with open(FILE_PATH, "r") as
            file:
            return json.load(file).

    except(FileNotFoundError, json.
            JSONDecodeError):
            return[]
```

PAGE NO.....
DATE / /

1) what is the work of strip() here?

Purpose of strip()

→ The strip() method removes leading and trailing whitespace from a string.
→ whitespace includes spaces tab (\t) and newlines (\n)

why is it used here?
→ In the line user_website = input("Please Enter Your Websites Name.").strip().strip() ensure that any accidental spaces at the beginning or end of the user's input are removed.

→ This prevents issues where the user might accidentally add spaces, which could cause problems when saving or comparing the input later.

Example:

```
user_input = "   example.com   "
print(user_input.strip()) # Output = "example.com"
```

2) what does .replace(" ", "").isalpha() do?

(1) .replace(" ", "")

→ This method replaces all spaces in the string with the empty strings ("") effectively removing all spaces from the string.
→ For example

```
user_input = "John Doe"
print(user_input.replace(" ", ""))
#output = 'JohnDoe'
```

(2) .isalpha()

→ The isalpha() method checks if all characters in the string are alphabetic (letters a-z or A-Z).

→ It returns [True] if the string contains only letters, otherwise [False]

Combined effect

- user_name.replace(" ", "").isalpha.
  → 1st, it removes all spaces from user_name.
  → Then, it checks if the resulting string contains only alphabetic characters.

(3) (char in special_chars for char in user_password)

(1) Iteration (for char in user_password).

→ This part loops through each character (char) in the string [user_password]

(2) Condition (char in special_chars).

→ This checks whether the current [char] is in [special_chars], which is a string containing all special characters from [string.punctuation] (e.g. !@#$...())

(4) "".join(random.choice(all_chars) for _ in range(password_length))

what is [choice]?

→ [choice] is method (function) from Python's built-in [random] module.

How it works?

→ random.choice(sequence) selects one random element from a given sequence.

→ In your case, `all_chars` is a string (which is a sequence of characters), so random.choice(all_chars) picks one random character from it.

(1) random.choice(all_chars):

→ selects a random character from `all_chars`, which is a combination of lowercase letters, uppercase letters, digits, and special characters.

(2) For _ in range(password_length):

→ This runs the loop `password_length` times (in this case `12` times).

→ The `_` is a throwaway variable, meaning we don't care about the index. (no of iterations)

~~for i in range(5)~~

↳ $.

→ Understanding index in loops.

```
for i in range(5)
    print(f"Iteration {i}: Hello!")
```

output

```
Iteration 0: Hello!
Iteration 1: Hello!
Iteration 2: Hello!
Iteration 3: Hello!
Iteration 4: Hello!
Iteration 5: Hello!
```

(3) `''.join(...)`

→ This method combines all randomly selected characters into a single string.

→ `''.join(...)` means that each randomly selected character is joined together without spaces.

PAGE NO.
DATE / /

Break vs Continue vs Return

→ break : Exits the loop.

→ Continue : skips the current iteration and moves to the next.

→ return : Exits the function and optionally return a value.

(5) (a) passwords.remove(queryname) is executed already and its already erase from memory and how that possible to access the removed data gain in (b) print.

1) How remove() works?

→ The remove() method removes the 1st occurence of the specified value (queryname) from the list (passwords)

→ However, the variable queryname still holds a reference to the that was removed from the list.

2) Memory Management in Python.

→ In Python, when you remove an object from a list (or any collection), the object is not immediately cleared from memory.

→ Instead, Python uses a reference-counting mechanism to manage memory. The object is only deleted from memory when there are no more references to it.

3) Why queryname still works after removal)

→ When you call passwords.remove(queryname), the dictionary is removed from the passwords list. but the variable queryname still holds a reference to the dictionary.

→ As long as ~~is still~~ queryname is still in scope (i.e. the function hasn't finished executing), the dictionary it reference is not deleted from memory

4) Accessing queryname after Removal.

→ After passwords.remove(queryname), you can still access the dictionary using the queryname variable because it still holds a reference to the dictionary

→ This is why you can print queryname['password'] and queryname['username'] even after the removing the dictionary from the list.